

บทที่ 2

ทฤษฎีพื้นฐานที่ใช้ในการวิจัยการทดสอบแบบ White-box

Testing และ Black-box Testing

ในหัวข้อนี้จะกล่าวถึงทฤษฎีพื้นฐานต่างๆ ที่เกี่ยวข้องในการวิจัย และพื้นฐานของวิธีการทดสอบทั้งแบบ White-box testing และ Black-box testing ซึ่งเนื้อหาในบทนี้จะกล่าวถึงเทคนิคของการวิเคราะห์ค่าต่างๆ คุณลักษณะของโครงสร้างของโปรแกรมที่จะทำการทดสอบ และพื้นฐานของภาษาที่จะทำการทดสอบ ซึ่งเนื้อหาทั้งหมดนี้จำเป็นสำหรับการศึกษา และประเมินประสิทธิภาพของการทำการทดสอบแบบ White-box testing และ Black-box testing

2.1 แนวความคิดเกี่ยวกับการทดสอบ

ทั้ง Black-box และ White-box คือรูปแบบการทดสอบ โปรแกรมรูปแบบหนึ่ง การทดสอบแบบ black box เป็นการทดสอบที่กระทำกับระบบแบบ “black-box” ดังนั้นจึงไม่จำเป็นที่จะต้องรู้เกี่ยวกับโครงสร้างภายใน ซึ่งปกตินี้จะถูกจำกัดความถึงการที่มุ่งไปทดสอบ requirement ต่างๆ โดยความหมายที่เหมือนคำว่า black-box นั้นรวมถึง behavioral functional opaque-box และ closed-box

White-box test design นั้นยอมให้มีการเข้าถึงส่วนข้างในของ “box” และยังมุ่งประเด็นไปในการใช้ความรู้เกี่ยวกับโครงสร้างภายในของ software ในการพิจารณาข้อมูลที่จะนำมาทดสอบ ค่าที่มีความหมายเดียวกันมีดังนี้ structural glass-box และ clear-box

ในขณะที่ White-box และ Black-box เป็นวิธีการที่นิยมใช้ ยังมีหลายๆคนที่นิยมใช้ Behavioral และ Structural , Behavioral test design มีข้อแตกต่างเล็กน้อยจาก Black-box testing design เนื่องจากการใช้ความรู้เกี่ยวกับโครงสร้างภายในนั้น ไม่ได้เป็นข้อห้ามที่เคร่งครัดมากนัก เพียงเป็นสิ่งที่ไม่เป็นที่เห็นด้วยเท่าไรนัก ในทางปฏิบัติ มันเป็นไปได้เป็นผลดีเท่าไรนักหากใช้การทดสอบเพียงแค่แบบเดียว บางคนใช้การผสมผสานในหลายๆวิธี ในการทดสอบ ซึ่งส่งผลให้เขาไม่ได้ถูกหยุดยั้งโดยข้อจำกัดของการใช้วิธีการเดียวเท่านั้น ซึ่งบางครั้งวิธีการแบบนี้จะถูกเรียกว่า “gray-box” หรือ “translucent-box” test design

เป็นสิ่งจำเป็นอย่างยิ่งที่จะทำความเข้าใจว่าวิธีการต่างๆถูกใช้การในขั้นตอน Test design phase และผลที่เกิดขึ้นคือ ขาดที่จะมองดูการทดสอบขณะที่มันทำงานอยู่

หมายเหตุไว้ว่า ทุกๆระดับของการทดสอบ (Unit testing, system testing และอื่นๆ) สามารถใช้วิธีการทดสอบได้หลายๆวิธี โดย Unit testing มักถูกทดสอบโดยวิธีการ Structural test design แต่นั่นก็มีสาเหตุมาจากที่ผู้ทดสอบไม่มีคำอธิบายของ Requirement ที่เป็นของ Unit level ที่มีเหตุผลเพียงพอ

ในการทดสอบโปรแกรมสามารถแบ่งระดับ (Level) ของการทดสอบออกได้เป็นระดับต่างๆ ดังนี้

2.1.1 การทดสอบระดับหน่วย (Unit testing)

เป็นการทดสอบการทำงานของแต่ละโมดูล (Module) ภายในโปรแกรม กรณีทดสอบจะสร้างขึ้นเพื่อทดสอบการทำงานภายในโมดูลนั้น โดยแต่ละโมดูลถูกทดสอบแยกจากกันทำให้ในบางโมดูลจำเป็นต้องใช้โปรแกรมที่ถูกเขียนขึ้นมาเฉพาะ นั่นคือตัวขับ (Driver) และตัวดำเนินการ (Stub) โดยตัวขับเป็นโปรแกรมที่ใช้จำลองการเรียกโมดูลที่ต้องการทดสอบพร้อมกับให้ข้อมูลเข้า (Input) และรับผลลัพธ์ (Output) จากการทำงานของโมดูลนั้น และตัวดำเนินการเป็นโปรแกรมจำลองที่โมดูลที่จะทดสอบนั้นต้องการเรียกใช้ ซึ่งกรณีทดสอบของการทดสอบระดับหน่วยต้องระบุตัวขับ และตัวดำเนินการ สำหรับแต่ละโมดูลไว้

2.2.2 การทดสอบการรวม (Integration testing)

เป็นการตรวจสอบข้อผิดพลาดที่เกิดจากการรวมโมดูลเข้าด้วยกัน และเป็นการตรวจสอบความเข้ากันได้ระหว่างโมดูล โดยมีวิธีการในการรวม ดังนี้

2.2.2.1) การรวมจากบนลงล่าง (Top-down approach) เป็นการรวมโมดูลที่อยู่ระดับบนสุดในแผนภาพ

โครงสร้าง (Structure Chart) ก่อน ทำให้ต้องมีการสร้างตัวดำเนินการเพื่อให้โมดูลที่รวมเข้าด้วยกันสามารถเรียกใช้ได้ ซึ่งในกรณีทดสอบต้องระบุ ตัวดำเนินการที่ส่วนที่ทดสอบนั้นเรียกไว้ด้วย

2.2.2.2) การรวมจากล่างขึ้นบน (Bottom-up approach) เป็นการรวมโมดูลที่อยู่ระดับล่างสุดในแผนภาพ

โครงสร้างก่อน ทำให้ต้องมีการสร้างตัวขับ เพื่อเรียกและส่งข้อมูลไปยังโมดูลที่รวมเข้าด้วยกัน ในกรณีทดสอบต้องระบุตัวขับ ที่เรียกส่วนที่ทดสอบนั้นด้วย

2.2.2.3) การรวมแบบแซนด์วิช (Sandwich approach) เป็นการรวมโมดูลที่อยู่ด้านบนด้วยวิธีการรวม

จากบนลงล่าง และรวมโมดูลที่อยู่ด้านล่างด้วยวิธีการแบบล่างขึ้นบน จากนั้นจึงรวมทั้ง 2 ส่วนเข้าด้วยกัน ต้องมีการสร้างทั้งตัวขับ และตัวดำเนินการ เพื่อให้โมดูลที่รวมกันนั้นเรียกใช้

2.2.2.4) การรวมแบบโมดูลวิกฤต (Critical Module approach) โดยเริ่มจากการรวมโมดูลที่มีความสำคัญมากกว่าโมดูลอื่น ๆ ก่อน เพื่อให้โมดูลเหล่านั้นได้ถูกทดสอบมากขึ้น

2.2.2.5) การรวมแบบบิกแบง (Big-bang approach) เป็นการรวมโมดูลเข้าด้วยกันในครั้งเดียว ซึ่งเป็น

วิธีที่นิยมใช้ในกรณีที่มีระยะเวลาในการทดสอบจำกัด

2.2.3 การทดสอบย้อนกลับ

เป็นการทดสอบที่สำคัญหลังจากการแก้ไขซอฟต์แวร์ โดยแบ่งออกเป็น 2 ประเภทคือ

2.2.3.1) การทดสอบหลังจากที่แก้ไขข้อผิดพลาดเรียบร้อยแล้ว เพื่อตรวจสอบการทำงานของซอฟต์แวร์ที่ได้ทำการ แก้ไข ว่าทำงานถูกต้องตามที่ได้ออกแบบไว้หรือไม่

2.2.3.2) การทดสอบหลังจากที่มีการเปลี่ยนแปลงหรือแก้ไขโปรแกรม เพื่อให้มั่นใจได้ว่าการทำงานของแต่ละโมดูลในโปรแกรมยังคงไม่มีข้อผิดพลาด และถูกต้องตามที่ได้ออกแบบไว้

2.2.4 การทดสอบระบบ (System testing)

เป็นการตรวจสอบความถูกต้องในการทำงานของส่วน (Element) ต่าง ๆ ในระบบทั้งหมด โดยกรณีทดสอบที่สร้างขึ้นต้องครอบคลุมทุกๆหน้าที่ (Function) การทำงานของระบบทั้งซอฟต์แวร์ และฮาร์ดแวร์ เพื่อให้ระบบที่พัฒนาไม่เกิดความผิดพลาดขึ้นในการทำงาน

2.2.4.1) การทดสอบตามหน้าที่ (Functional) โดยพยายามทดสอบความต้องการของผู้ใช้ ทั้งหมดที่ได้กำหนดไว้ในข้อกำหนดความต้องการซอฟต์แวร์ (Software requirement specification)

2.2.4.2) การทดสอบอื่น ๆ (Non-Functional) ประกอบด้วย

2.2.4.2.1) การทดสอบความกดดัน (Stress testing) โดยทดสอบระบบภายใต้สภาพการใช้งานที่สูงมากๆ มีจุดประสงค์เพื่อต้องการทราบความสามารถสูงสุดของระบบภายใต้ทรัพยากรที่กำหนด เช่น การทดสอบจำนวนรายการที่เกิดขึ้น (Transaction) มากที่สุดที่ระบบสามารถรองรับได้ เป็นต้น

2.2.4.2.2) การทดสอบประสิทธิภาพ (Performance testing) เพื่อแสดงว่าระบบมีประสิทธิภาพตามที่กำหนดไว้หรือไม่ และเพื่อปรับแต่งระบบให้มีประสิทธิภาพดีขึ้น โดยตรวจสอบจากเวลาตอบสนอง (Response time) ปริมาณงาน (Throughput) การใช้งานหน่วยประมวลผลกลาง (CPU utilization) เป็นต้น

2.2.4.2.3) การทดสอบส่วนหลัง (Background testing) เพื่อแสดงความสามารถของระบบในกรณีที่รองรับรายการมากกว่าหนึ่งรายการในเวลาเดียวกัน

2.2.4.2.4) การทดสอบโครงแบบ (Configuration testing) เพื่อต้องการทราบข้อกำหนดของฮาร์ดแวร์ (Hardware) ที่เหมาะสมกับระบบ

2.2.4.2.5) การทดสอบการกู้ระบบ (Recovery testing) เพื่อแสดงว่าระบบสามารถกู้ข้อมูลกลับคืนมาได้หากระบบเกิดความเสียหายขึ้น

2.2.4.2.6) การทดสอบความปลอดภัย (Security testing) เพื่อแสดงว่าระบบมีประสิทธิภาพเพียงพอในการรักษาความปลอดภัย และป้องกันระบบจากการลักลอบ ใช้งานของผู้ที่ไม่ได้รับอนุญาต

ข้อเปรียบเทียบระหว่างการทดสอบแบบระดับหน่วย (Unit testing) กับ การทดสอบแบบการรวม (Integration testing)

การทดสอบระดับหน่วย (Unit testing)

- เป็นการทดสอบในส่วนประกอบของโปรแกรมในแต่ละส่วน
- เป็นการทดสอบเพื่อดูถึงหน้าที่ความรับผิดชอบเฉพาะในส่วนของโปรแกรมที่ทดสอบเท่านั้น
- การทดสอบในระดับนี้นั้นต้องอาศัยประสบการณ์ของผู้พัฒนาโปรแกรมเพื่อดูความผิดพลาดของโปรแกรมที่จะเกิดขึ้น

การทดสอบแบบรวม (Integration testing)

- เป็นการทดสอบโดยเอาส่วนประกอบของโปรแกรมทั้งหมดมาทดสอบเพื่อทำงานร่วมกันว่าเกิดข้อผิดพลาดใดขึ้นบ้าง

- การทดสอบเป็นยูนิตหลักคุณสมบัติเฉพาะคุณสมบัติเฉพาะระบบ ซึ่งเป็นอินเตอร์เฟซในการออกแบบ

2.2 เทคนิคที่ใช้ในการออกแบบกรณีทดสอบ

2.2.1 การออกแบบกรณีทดสอบแบบไวท์บ็อกซ์ (White-box testing)

เป็นการสร้างกรณีทดสอบโดยพิจารณาจาก โครงสร้าง (Structure) ของโปรแกรม โดยสร้างกรณีทดสอบที่ครอบคลุมทุกเส้นทาง (Path) ตามโครงสร้างของโปรแกรม เพื่อให้แน่ใจว่าทุกคำสั่งในโปรแกรมได้รับการทดสอบอย่างน้อยหนึ่งครั้ง และทุกคำสั่งที่เป็นเงื่อนไขถูกทดสอบ ทั้งกรณีที่เงื่อนไขนั้นเป็นจริงและกรณีที่เงื่อนไขนั้นเป็นเท็จ นอกจากนี้ยังสามารถค้นพบข้อผิดพลาดที่เกิดขึ้นจากข้อมูลภายในโมดูลได้อีกด้วย

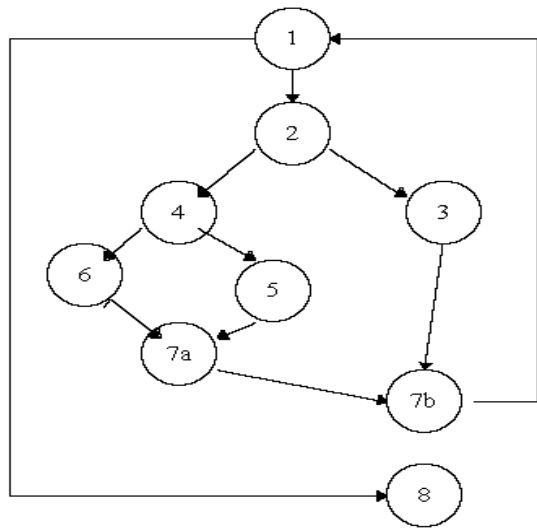
การสร้างกรณีทดสอบด้วยวิธีการของไวท์บ็อกซ์ ผู้ทดสอบต้องสร้างกราฟการควบคุมกระแส (Control flow graph) จากโปรแกรมที่พัฒนาขึ้น และนำกราฟที่ได้มาสร้างกรณีทดสอบเพื่อให้ครอบคลุมทุกคำสั่ง ทุกเส้นทาง และทุกเงื่อนไขของโปรแกรม ดังรูปที่ 2.1 แสดงรหัสเทียม (Pseudocode) และกราฟการควบคุมกระแสที่สร้างขึ้นจากรหัสเทียมนั้น ซึ่งสามารถเขียนเส้นทางที่ได้รับการทดสอบดังต่อไปนี้

- 1) 1, 8
- 2) 1, 2, 3, 7b, 1, 8
- 3) 1, 2, 4, 5, 7a, 7b, 1, 8
- 4) 1, 2, 4, 6, 7a, 7b, 1, 8

```

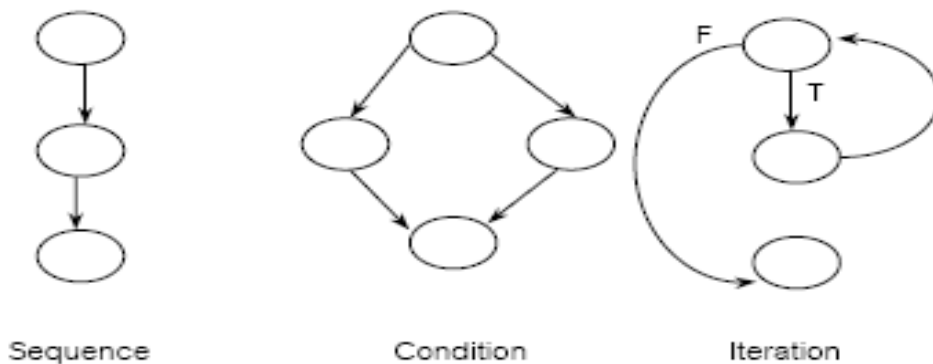
PDL for PROCEDURE SORT
1:   do while records remain
      read record;
2:   if record field 1 = 0
3:     then process record;
          store in buffer;
          increment counter;
4:   elsif record field 2 = 0
5:     then reset record;
6:     else process record;
          store in file;
7a:  endif
7b:  enddo
8:  end

```



รูปที่ 2.1 ตัวอย่างรหัสเทียม (Psuedocode) และกราฟการควบคุมกระแสที่สร้างขึ้นจากรหัสเทียม

ซึ่งเราสามารถแบ่งกราฟควบคุมกระแส (Control Flow Graph) ตามเงื่อนไขของโปรแกรมได้ออกเป็นหลักๆ ได้ 3 ประเภทดังรูปที่ 2.2



รูปที่ 2.2 กราฟควบคุมกระแส (Control Flow Graph of 3 Prime Structures)

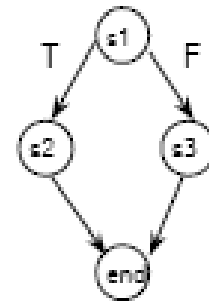
Branch Coverage

คือ กราฟควบคุมกระแสของโปรแกรมเงื่อนไขจากเงื่อนไขหนึ่งที่สามารถรับ ข้อมูลเข้ามาแล้วสามารถทำการเลือกเส้นทางการทำงานไปตามเงื่อนไขที่โปรแกรมได้กำหนดไว้ ซึ่งถ้าข้อมูล

ที่รับเข้ามาถูกต้องตามเงื่อนไขก็จะไปทำต่อในอีกเงื่อนไขหนึ่ง ถ้าข้อมูลที่ได้รับเข้ามาผิดจากเงื่อนไขที่กำหนดไว้ก็จะไปทำต่อในอีกเงื่อนไขหนึ่ง

ดังเช่นตัวอย่างที่จะแสดงให้เห็นถึงโปรแกรมเงื่อนไขที่เป็นประเภท Condition ดังรูปที่ 2.3

```
function check (x:integer) boolean
begin
s1   if (x >=0) and (x <= 100)
s2   then check = true
s3   else check = false
end
```



test case $x = 5$ and $x = -5$ satisfy branch coverage

รูปที่ 2.3 ตัวอย่างโปรแกรมเงื่อนไขแบบ Branch Coverage

Statement Coverage

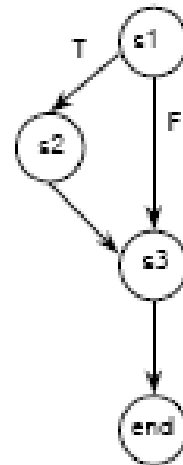
คือ กราฟควบคุมกระแสของโปรแกรมเงื่อนไขที่เป็นเงื่อนไขที่เลือกเส้นทางการทำงานต่อตามข้อมูลที่รับเข้ามาว่าถ้าข้อมูลที่รับเข้ามาถูกต้องตามเงื่อนไขก็จะไปทำยังเงื่อนไขถัดไปแล้วกลับมาทำงานต่อยังเงื่อนไขในเส้นทางเดิม แต่ถ้าข้อมูลที่รับมาไม่ถูกต้องตามเงื่อนไขก็จะข้ามไปทำต่อยังเงื่อนไขในเส้นทางเดิมต่อไปเลย

ดังเช่นตัวอย่างที่แสดงให้เห็นถึงโปรแกรมที่เป็นเงื่อนไขประเภท Iteration ดังที่แสดงไว้ในรูปที่ 2.4

```

function abs (x:int): integer
begin
s1   if x < 0 then
s2     x := 0-x
s3     abs := x
end

```



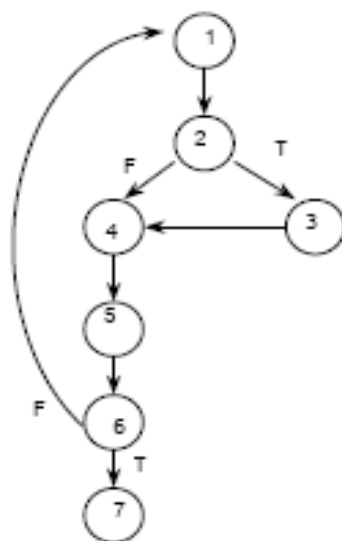
test case x = -1 satisfy statement coverage

รูปที่ 2.4 ตัวอย่างโปรแกรมเงื่อนไขแบบ Statement Coverage

Path Coverage

คือ การหาเส้นทางการทำงานของโปรแกรมที่สามารถเป็นไปได้ทั้งหมด ที่จะทำให้โปรแกรมสามารถทำงานได้จนจบการทำงาน

ดังตัวอย่างในรูปที่ 2.5 จะแสดงให้เห็นถึงเส้นทางการทำงานจากกราฟควบคุมกระแส ว่าการทำงานของโปรแกรมนั้นสามารถเดินทางผ่านไปในช่วงใดได้บ้าง โปรแกรมจึงจะทำงานได้



Statement coverage

1-2-3-4-5-6-7

Branch coverage

A. 1-2-3-4-5-6-7

B. 1-2-4-5-6-1

Path coverage

(all possible path)

A. 1-2-3-4-5-6-7

B. 1-2-3-4-5-6-1

C. 1-2-4-5-6-7

D. 1-2-4-5-6-1

รูปที่ 2.5 แสดงการหาเส้นทางการทำงานจากกราฟควบคุมกระแส

โปรแกรมตัวอย่างเพื่อทำการตรวจหาเส้นทางการทำงานของโปรแกรม

An implementation of getNumDaysInMonth() method

```
public class MonthOutOfBounds extends Exception {...};
```

```
public class YearOutOfBounds extends Exception {...};
```

```
class MyGregorianCalendar
```

```
{    public static boolean isLeapYear(int year)
```

```
{    boolean leap;  
    if (year%4)  
    {    leap = true;  
    }  
    else
```

```
{    leap = false;  
  
    }  
  
    return leap;  
  
}
```

```
public static int getNumDaysInMonth(int month, int year) throws MonthOutOfBounds,  
YearOutOfBounds
```

```
{    int numDays;  
    if (year < 1)  
    {    throw new YearOutOfBounds(year);
```

```
}
```

```
if (month == 1 || month == 3 || month == 5 || month == 7 || month == 10 || month == 12)
```

```
{    numDays = 32;
```

```
}
```

```
else if (month == 4 || month == 6 || month == 9 || month == 11)
```

```
{    numDays = 30;
```

```
}
```

```
else if (month == 2)
```

```
{    if (isLeapYear(year))
```

```
        {    numDays = 29;
```

```
        }
```

```
else
```

```
{    numDays = 28;
```

```
}
```

```
}
```

```
else
```

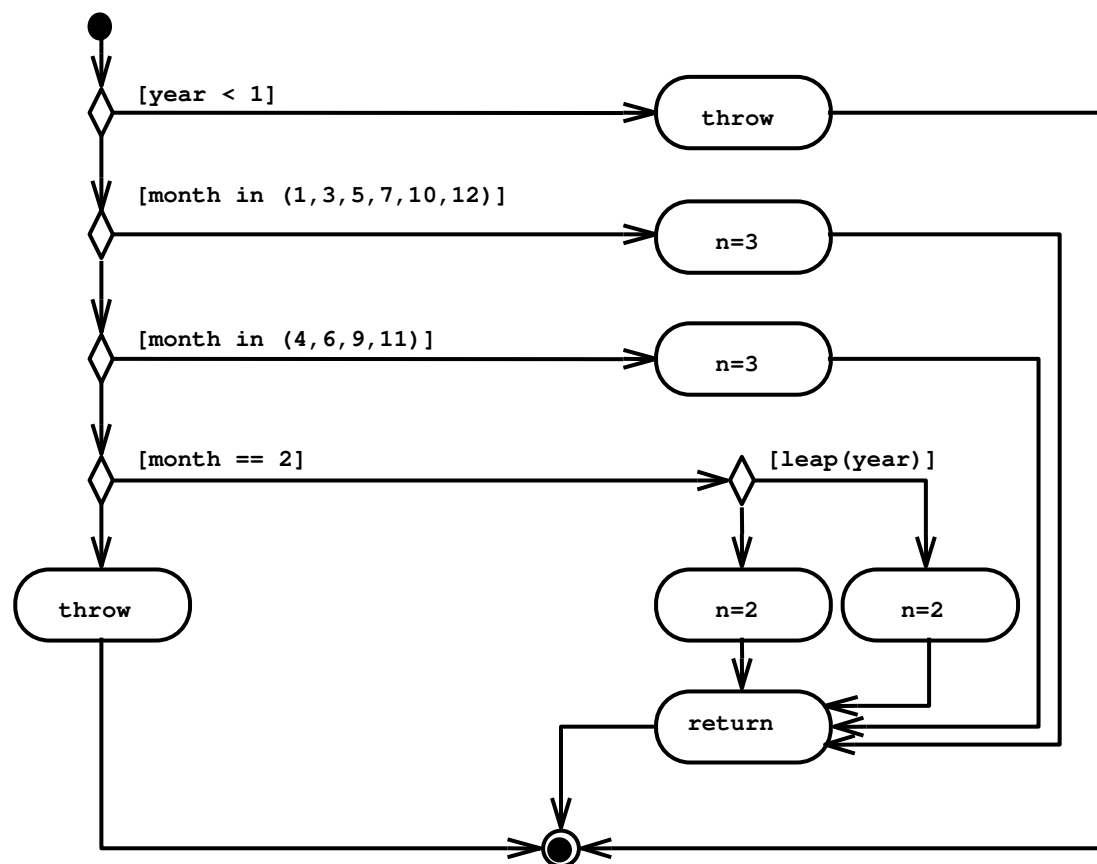
```
{    throw new MonthOutOfBounds(month);
```

```
}
```

```
return numDays;
```

```
}
```

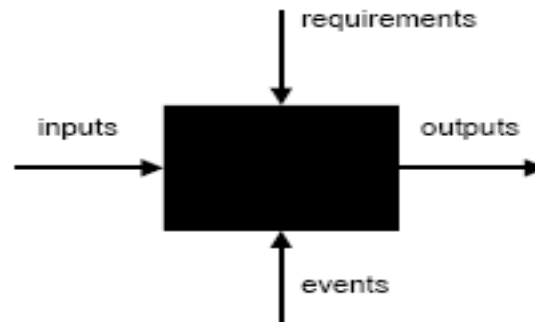
Path testing



รูปที่ 2.6 แสดงการวิเคราะห์ Path Testing จากโปรแกรม

2.2.2 การออกแบบกรณีทดสอบแบบแบล็กบ็อกซ์ (Black-box testing)

เป็นการสร้างกรณีทดสอบโดยพิจารณาจากข้อกำหนด (Specification) และเงื่อนไข (Condition) ของข้อมูลนั้น การทดสอบสนใจเฉพาะข้อมูลเข้าและผลลัพธ์ของโมดูลที่ต้องการทดสอบ เพื่อให้ตรงตามข้อกำหนดที่ได้ออกแบบไว้เท่านั้น ซึ่งจะไม่พิจารณาถึงโครงสร้างภายในโมดูลนั้น

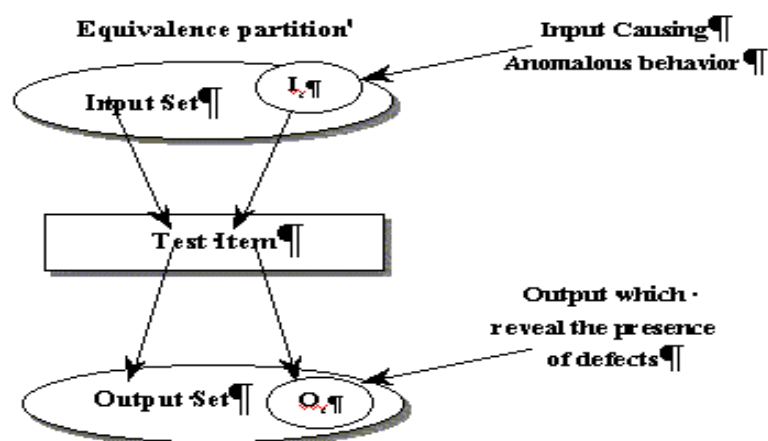


รูปที่ 2. 7 แสดงให้เห็นถึงวิธีการทดสอบแบบแบล็กบ็อกซ์

การออกแบบกรณีทดสอบแบบแบล็กบ็อกซ์มีขั้นตอนที่สำคัญ 2 ขั้นตอน คือ

2.2.2.1) การแยกชั้นสมมูล (Equivalence class partitioning)

เป็นเทคนิคเพื่อใช้ตัดสินว่าชั้นของข้อมูลเข้า (Input data) ใดเป็นชั้นปกติ และชั้นใดเป็นชั้นที่ทำให้เกิดข้อผิดพลาด ดังรูปที่ 2.2 แสดงส่วนสมมูล (Equivalence partition) ของข้อมูลเข้า ซึ่งประกอบด้วยชั้นของข้อมูลเข้าที่ปกติ (Input Set) เป็นชั้นของข้อมูลเข้าที่เมื่อเข้าสู่รายการทดสอบแล้วให้ผลลัพธ์อยู่ในเซตของผลลัพธ์ที่ถูกต้อง (Output Set) และชั้นของข้อมูลเข้าที่ไม่ถูกต้อง (I_e) ที่เมื่อเข้าสู่รายการทดสอบแล้วให้ผลลัพธ์อยู่ในเซตของผลลัพธ์ที่ไม่ถูกต้อง (O_e) ซึ่งจะแสดงให้เห็นถึงข้อผิดพลาดของรายการทดสอบนั้น



รูปที่ 2.8 แสดงส่วนสมมูลของข้อมูลเข้า ประกอบด้วยชั้นของข้อมูลเข้าที่ปกติ และชั้นของข้อมูลเข้าที่ทำให้เกิดข้อผิดพลาด

การแยกประเภทของส่วนสมมูลสามารถทำได้โดยพิจารณาจากข้อกำหนด และเงื่อนไขของข้อมูลเข้า จากเอกสารของผู้ใช้งาน (User Documentation) หรือจากประสบการณ์ของผู้ทดสอบเองว่าชั้นของข้อมูลเข้าใดเป็นชั้นที่จะทำให้เกิดข้อผิดพลาดขึ้นในโปรแกรม และชั้นของข้อมูลเข้าใดเป็นชั้นของข้อมูลที่ต้อง

ผลที่ได้จากขั้นตอนของการแยกชั้นของข้อมูลเข้า (Input equivalence class) คือชั้นของข้อมูลเข้าที่ถูกต้อง (Valid class) และชั้นของข้อมูลเข้าที่ทำให้เกิดข้อผิดพลาด (Invalid class) บางครั้งอาจเรียกว่าชั้นของข้อมูลเข้าเหล่านั้นว่า ชั้นสมมูล (Equivalence class) สำหรับข้อกำหนดและเงื่อนไขของข้อมูลเข้านั้น

ข้อกำหนดหรือเงื่อนไขของข้อมูลเข้าที่พบโดยทั่วไป ได้แก่

- ข้อมูลที่ระบุค่าเฉพาะ
- ข้อมูลที่ระบุค่าอยู่ในช่วงของข้อมูล
- ข้อมูลที่ระบุเขตของข้อมูล
- ข้อมูลประเภทค่าตรรกศาสตร์

ซึ่งสามารถกำหนดชั้นสมมูลสำหรับแต่ละข้อกำหนดหรือเงื่อนไขของข้อมูล ได้ดังนี้

1. ถ้าข้อมูลเข้าระบุค่าเฉพาะ จะมีหนึ่งชั้นสมมูลที่เป็นชั้นของข้อมูลเข้าที่ถูกต้อง คือข้อมูลเข้าที่มีค่าเท่ากับค่าเฉพาะที่ระบุ และ สองชั้นของข้อมูลเข้าที่ทำให้เกิดข้อผิดพลาด คือข้อมูลที่มีค่ามากกว่า และ น้อยกว่าค่าเฉพาะที่ระบุ ตัวอย่างเช่น ถ้าข้อมูลเข้ากำหนดให้มีค่าเท่ากับ 1 จะมี 1 ชั้นสมมูลที่เป็นชั้นของข้อมูลเข้าที่ถูกต้อง คือชั้นสมมูลของข้อมูลเข้าที่มีค่าเท่ากับ 1 และมี 2 ชั้นสมมูลที่เป็นชั้นสมมูลของข้อมูลเข้าที่ไม่ถูกต้อง คือชั้นสมมูลของข้อมูลเข้าที่มีค่าน้อยกว่า 1 และชั้นสมมูลของข้อมูลเข้าที่มีค่ามากกว่า 1

2. ถ้าข้อมูลเข้าระบุค่าอยู่ในช่วงของข้อมูล จะมีหนึ่งชั้นสมมูลที่เป็นชั้นของข้อมูลเข้าที่ถูกต้อง คือข้อมูลที่มีค่าอยู่ในช่วงนั้น และ สองชั้นสมมูลของข้อมูลเข้าที่ไม่ถูกต้อง คือข้อมูลที่มีค่ามากกว่า และ น้อยกว่าช่วงของข้อมูลนั้น เช่น ถ้าข้อมูลเข้ามีค่าอยู่ระหว่าง 0 ถึง 10 จะมี 1 ชั้นสมมูลที่เป็นชั้นของข้อมูลเข้าที่ถูกต้อง คือชั้นสมมูลของข้อมูลเข้าที่มีค่าอยู่ระหว่าง 0 ถึง 10 และมี 2 ชั้นสมมูลที่เป็นชั้นสมมูลของข้อมูลเข้าที่ทำให้เกิดข้อผิดพลาด คือชั้นสมมูลของข้อมูลเข้าที่มีค่าน้อยกว่า 0 และชั้นสมมูลของข้อมูลเข้าที่มีค่ามากกว่า 10

3. ถ้าข้อมูลเข้าที่ระบุเขตของข้อมูล จะมีหนึ่งชั้นสมมูลที่เป็นชั้นของข้อมูลเข้าที่ถูกต้อง คือข้อมูลที่เป็นสมาชิกของเขตที่ระบุ ที่ไม่ถูกต้อง คือข้อมูลที่ไม่เป็นสมาชิกของเขตที่ระบุ เช่น ถ้าข้อมูลเข้าเป็นเขตของจำนวนเฉพาะ จะมี 1 ชั้นสมมูลที่เป็นข้อมูลเข้าที่ถูกต้องคือข้อมูลที่เป็นเลขจำนวนเฉพาะ และมี 1 ชั้นสมมูลที่เป็นชั้นสมมูลของข้อมูลเข้าที่ทำให้เกิด ข้อผิดพลาด คือข้อมูลที่ไม่เป็นเลขจำนวนเฉพาะ

4. ถ้าข้อมูลเข้าประเภทค่าตรรกศาสตร์ จะมีหนึ่งชั้นสมมูลที่เป็นชั้นของข้อมูลเข้าที่ถูกต้อง และ หนึ่งชั้นของข้อมูลเข้าที่ไม่ถูกต้อง เช่นข้อมูลเข้าที่กำหนดค่าของข้อมูลเข้าคือ จริง ชั้นสมมูลที่เป็นข้อมูลเข้าที่ถูกต้องคือข้อมูลที่เป็นมีค่าความจริงเป็นจริง และมี 1 ชั้นสมมูลที่เป็นชั้นสมมูลของข้อมูลเข้าที่ทำให้เกิดข้อผิดพลาด คือข้อมูลที่มีค่าความจริงเป็นเท็จ

2.2.2.2 การวิเคราะห์ค่าขอบเขต (Boundary Value Analysis)

เนื่องจากข้อผิดพลาดที่เกิดขึ้นในโปรแกรมส่วนใหญ่จะเกิดในบริเวณที่เป็นค่าขอบเขตของข้อมูล ทำให้เมื่อเราสร้างชั้นสมมูลแล้วต้องมีการทำขั้นตอนการวิเคราะห์ค่าขอบเขต เพื่อให้กรณีทดสอบที่สร้างขึ้นมีประสิทธิภาพมากที่สุดในการค้นหาข้อผิดพลาด

รูปแบบของการวิเคราะห์ค่าขอบเขตสำหรับลักษณะเฉพาะ หรือเงื่อนไขของข้อมูลเข้าโดยทั่วไป

1. ถ้าข้อมูลเข้าระบุค่าอยู่ในช่วงของข้อมูล a และ b ข้อมูลสำหรับทดสอบค่าขอบเขตในกรณีทดสอบที่สร้างขึ้นควรมีค่าอยู่ เหนือและ ใต้ค่า a และ b เล็กน้อย

2. ถ้าข้อมูลเข้าระบุจำนวนมากที่สุด หรือน้อยที่สุดของข้อมูลเข้า กรณีทดสอบที่สร้างขึ้นควรทดสอบค่ามากที่สุด น้อยที่สุด นั้น และทดสอบบริเวณที่เป็นขอบเขต คือ เหนือและใต้ จำนวนมากที่สุด น้อยที่สุดนั้นด้วย

3. ประยุกต์ข้อ 1 และ 2 สำหรับ ผลลัพธ์ ที่ต้องการ

4. ถ้าโครงสร้างภายในของโปรแกรมมีการกำหนดขอบเขตไว้ สร้างข้อมูลทดสอบเพื่อทดสอบโครงสร้างนั้น เช่นถ้าโปรแกรมกำหนดข้อมูลแบบอาร์เรย์ขนาด 100ข้อมูล ต้องสร้างข้อมูลทดสอบในบริเวณที่เป็นขอบ (99,100,101) เขตสำหรับทดสอบข้อมูลอาร์เรย์ขนาดนั้น

2.2.2.3 การวางแผนการทดสอบ (Test Planning)

ระบุถึงขอบเขต แนวทาง ทรัพยากร และกำหนดการ ของการทดสอบ ระบุส่วนของโปรแกรมลักษณะ (Feature) ที่จะทดสอบ กิจกรรมที่ต้องทำ ความรับผิดชอบของผู้ทดสอบ และความเสี่ยงที่จะเกิดขึ้นในแผนการทดสอบ

2.2.2.4 รายละเอียดของการทดสอบ (Test Specification)

ประกอบด้วยเอกสารและข้อมูลต่าง ๆ ได้แก่

2.2.2.4.1 รายละเอียดเกี่ยวกับการออกแบบการทดสอบ (Test-design Specification) ระบุการออกแบบการทดสอบเพื่ออธิบายลักษณะที่จะทดสอบที่ได้กำหนดไว้ใน การวางแผนการทดสอบ นอกจากนั้นยังระบุถึงกรณีทดสอบที่เกี่ยวข้องและหลักในการพิจารณาการทดสอบในแต่ละกรณีทดสอบว่าผ่านหรือไม่ผ่านเกณฑ์ที่กำหนด

2.2.2.4.2 รายละเอียดเกี่ยวกับกรณีทดสอบ (Test-case Specification) ระบุค่าของข้อมูลเข้าเพื่อใช้ในการทดสอบและกำหนดผลลัพธ์ที่ต้องการ นอกจากนั้นยังระบุข้อจำกัดต่าง ๆ ที่มีในการใช้กรณีทดสอบนั้น

ข้อมูลในรายละเอียดเกี่ยวกับกรณีทดสอบ ได้แก่

- ตัวระบุกรณีทดสอบ (Test case specification identifier) เพื่อให้ผู้ทดสอบสามารถระบุกรณีทดสอบที่ใช้ทดสอบแต่ละรายการทดสอบได้

- รายการที่ทดสอบ (Test item) แสดงชื่อ และ/หรือ หน้าทีของรายการที่จะทดสอบสำหรับกรณีทดสอบนั้น

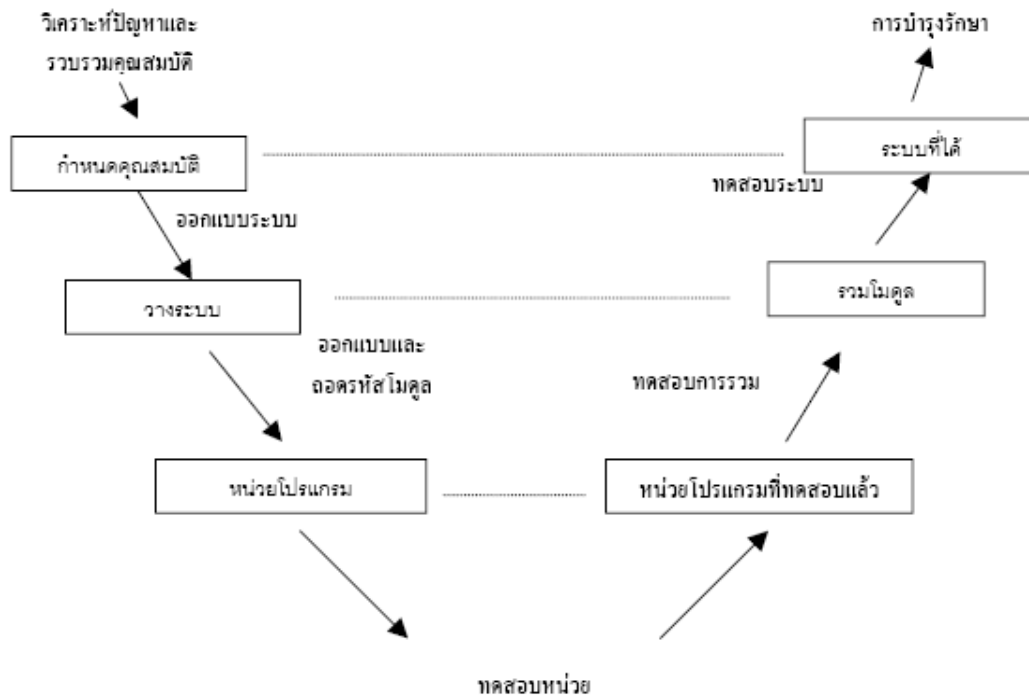
- ค่าของข้อมูลเข้าเพื่อใช้ในการทดสอบ (Input specification)

- ผลลัพธ์ที่คาดว่าจะได้รับการทดสอบ (Output specification)
- ลักษณะสิ่งแวดล้อมที่ต้องการในการทดสอบ (Environment needs) เช่น ลักษณะอุปกรณ์ซอฟต์แวร์
- ความต้องการพิเศษอื่นๆ (Special procedural requirement) ระบุความต้องการพิเศษอื่นๆ เพื่อใช้สำหรับกรณีทดสอบนี้
- รายการกรณีทดสอบที่เกี่ยวข้อง (Intercase dependency) แสดงรายการกรณีทดสอบที่ต้องทำการทดสอบก่อนหน้ากรณีทดสอบนี้ และความสัมพันธ์กับกรณีทดสอบนี้

2.3 ความสมเหตุสมผล การทวนสอบ และ การทดสอบ

ความสมเหตุสมผล (Validation) เกี่ยวข้องกับการตรวจสอบว่าเอกสารประกอบโปรแกรมและข้อมูลของโปรแกรมที่ผลิตขึ้นตรงตามข้อกำหนดของปัญหาการทวนสอบ (Verification) หมายถึง การทวนสอบว่าผลผลิตนั้นถูกต้อง สมบูรณ์ และ ผลที่ได้จากขั้นตอนนั้นเข้ากันได้กับขั้นตอนที่อยู่ก่อนหน้า มีความสมเหตุสมผลสามารถอธิบายได้ ในการทวนสอบใช้คำถามว่า เราผลิตสินค้าที่ถูกต้องหรือไม่ และการทดสอบใช้ตอบคำถามว่า เราใช้วิธีการผลิตที่ถูกต้องหรือไม่ ข้อผิดพลาดสามารถเกิดขึ้นในขั้นตอนต่างๆ ของวงจรการพัฒนาซอฟต์แวร์ เช่น ข้อกำหนด ของซอฟต์แวร์ อาจจะมีรายละเอียดพอที่จะสะท้อนถึงข้อมูลของปัญหาที่กำหนดได้ หรือของความ ต้องการลูกค้า

อัลกอริทึมอาจประกอบด้วยข้อผิดพลาดทางตรรกและหน่วยของโปรแกรมเขียนไว้ไม่ถูกต้อง การตรวจและการแก้ข้อผิดพลาด ซึ่งเป็นส่วนที่สำคัญของการพัฒนาซอฟต์แวร์ คือ การทดสอบความสมเหตุสมผลและการทวนสอบการทดสอบข้อผิดพลาดที่เกิดขึ้นในการพัฒนาโปรแกรมมีหลายวิธี เช่น การทดสอบยูนิต (unit test) โดยแยกทดสอบหน่วยของโปรแกรมแต่ละหน่วย การทดสอบเบ็ดเสร็จ (integration test) สามารถตรวจสอบ หน่วยเหล่านี้มีการรวมอย่างถูกต้อง การทดสอบระบบ คือการทดสอบว่าระบบมีการทำงานอย่างถูกต้อง สิ่งเหล่านี้รวมอยู่ในขั้นตอนต่างๆ ของวงจรพัฒนาซอฟต์แวร์และแสดงในแผนภาพของโมเดลวงจรชีวิตตัววี (V life cycle model)



รูปที่ 2.9 วงจรตัววีใช้สำหรับการทดสอบซอฟต์แวร์

2.4 ข้อดีและข้อเสียของการตรวจสอบโปรแกรมแบบ Black block testing

2.4.1 ข้อดีของการตรวจสอบโปรแกรมแบบ Black block testing

ในโปรแกรมที่มีขนาดใหญ่(มี code หลายๆบรรทัด) สามารถใช้งานได้อย่างมีประสิทธิภาพมากกว่าการทดสอบแบบ glass box

ผู้ทดสอบไม่จำเป็นต้องมีความรู้เกี่ยวกับโปรแกรมที่นำมาทดสอบ ซึ่งหมายความว่าถึงภาษาที่ใช้สร้างโปรแกรมขึ้นมาด้วย

ผู้ทดสอบกับโปรแกรมเมอร์เป็นอิสระต่อกันอย่างสิ้นเชิง

ผู้ทดสอบจะทดสอบโดยมีข้อคิดเห็นของผู้ใช้เป็นหลัก

สามารถช่วยอธิบายคุณสมบัติของโปรแกรมที่ไม่ชัดเจน และไม่สอดคล้องได้

สามารถออกแบบ Test case ได้ทันทีที่ทำการกำหนดคุณสมบัติของโปรแกรมเสร็จสิ้น

2.4.2 ข้อเสียของการทดสอบโปรแกรมแบบ Black box testing

สามารถกำหนด Input ได้ไม่มากนักในการทดสอบ การทดสอบที่ทดสอบทุกๆ Input เป็นไปได้ยาก

เนื่องด้วยการกำหนดคุณสมบัติที่ไม่ชัดเจนและรัดกุมของโปรแกรม ทำให้การออกแบบ Test case ทำได้ลำบาก

หากโปรแกรมเมอร์ไม่แจ้ง Test case ที่ได้ทำการทดสอบแล้ว จะทำให้ผู้ทดสอบเสียเวลาในการทดสอบกรณีอื่นๆ โดยไม่จำเป็น

การทดสอบแบบนี้อาจจะเหลือ Path ของโปรแกรมที่ไม่ได้ทดสอบอีกมาก

ไม่ได้มีการเข้าถึง Code จริงๆ ซึ่งหากเป็นโปรแกรมที่ซับซ้อนจะทำให้มี Error เกิดขึ้นได้มากกว่า

2.5 Backus Normal Form หรือ Backus Naur Form

Backus Normal Form นั้นเป็นวิธีการที่ใช้สำหรับอธิบาย Syntax ของภาษาเพื่อให้ครอบคลุมและเข้าใจได้ถูกต้อง และเพื่อกำหนดเป็นขอบเขตของรูปแบบในการเขียนโคดของภาษาในการระบุ Syntax ของโปรแกรมที่สามารถนำมาใช้ในการทดสอบของโปรแกรมทดสอบที่จะจัดทำขึ้นมานี้

ตัวอย่างรูปแบบของ Backus Normal Form หรือ Backus Naur Form

$\langle \text{Number} \rangle ::= \langle \text{Digit} \rangle \mid \langle \text{Number} \rangle \langle \text{Digit} \rangle .$

$\langle \text{Digit} \rangle ::= 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 .$

จากตัวอย่างเป็นการระบุรูปแบบของอธิบายว่า Number นั้นสามารถเขียนให้อยู่ในรูปของอะไรได้บ้าง นั่นก็คือสามารถเขียนให้อยู่ในรูปของ Digit หรือ กลับไปเป็น Number และมี Digit มาต่อเพิ่มได้อีก

และต้องทำการอธิบายถึง Digit ว่าคืออะไรด้วยในส่วนถัดมา ซึ่ง Digit ก็ได้อธิบายว่าเป็นตัวเลข 1 ตัว ตั้งแต่ 0 ถึง 9 นั่นเอง

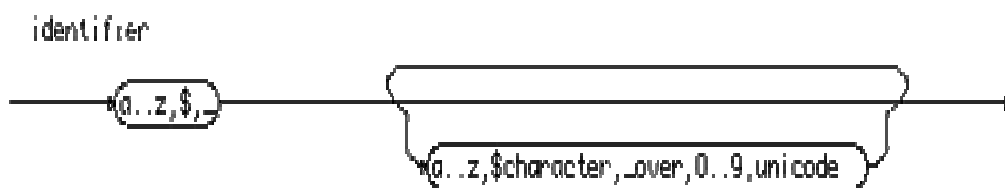
นั่นก็หมายความว่า Number ก็คือตัวเลขตั้งแต่ 1 ตัวขึ้นไป เช่น 0, 10, 101 ... เป็นต้น

ตัวอย่างของ Backus-Naur form ในภาษาซี

The syntax of C Program in Backus-Naur form

Identifier

<Identifier> ::= a..z | \$ | _ | < a..z | \$ | _ | 0..9 | unicode character over 00C0 >

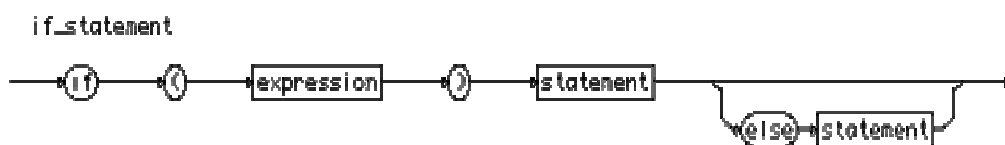


รูปที่ 2.10 รูปแสดง Backus-Naur form ของ Identifier ในภาษาซี

If statement

<If statement> ::= if < expression > then < statement >

| if < expression > then < statement > else <statement >



รูปที่ 2.11 รูปแสดง Backus-Naur form ของ If statement ในภาษาซี

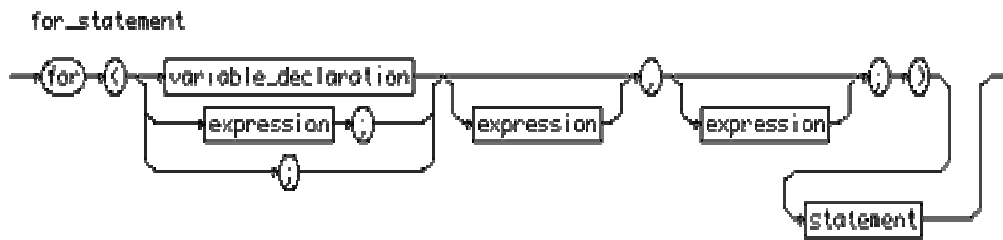
For statement

<For statement> ::= for (variable declaration | (<expression>) | ";")

<expression> ";"

<expression> ";"

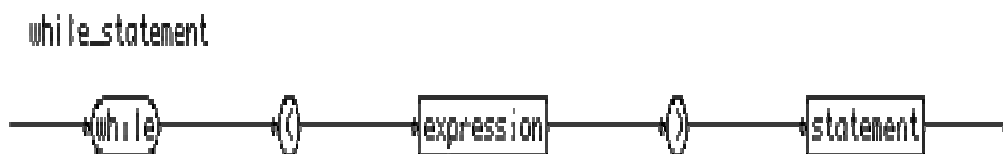
) < statement >



รูปที่ 2.12 รูปแสดง Backus-Naur form ของ For Statement ในภาษาซี

while_statement

<while_statement> ::= while (< expression >) < statement >



รูปที่ 2.13 รูปแสดง Backus-Naur form ของ While Statement ในภาษาซี

Expression

< expression > ::= < numeric_expression >

| < testing_expression >

| < logical_expression >

| < string_expression >

| < casting_expression >

| < null >

| < identifier >

| (< expression >)

2.6 งานวิจัยที่เกี่ยวข้อง

2.6.1) ระบบจัดการกรณีทดสอบ (Test case management System)

พัฒนาโดย Hiren D. Desai เป็นระบบที่พัฒนาขึ้นเพื่อใช้ในการทดสอบโปรแกรมในหน่วยงานของบริษัทเบลเซาท์ (BellSouth Co. Ltd.) ที่ประเทศสหรัฐอเมริกา ระบบจะเก็บรวบรวมข้อมูลกรณีทดสอบ และผลการทดสอบสำหรับโครงการต่างๆที่กำลังพัฒนา ผู้ทดสอบจะทำการบันทึกกรณีทดสอบที่ได้ออกแบบไว้ หลังจากนั้นระบบจะสร้างบทคำสั่งการทดสอบ (Test script) เพื่อนำไปทดสอบโปรแกรม ในขั้นตอนการทดสอบโปรแกรมระบบสามารถรับผลการทดสอบจากผู้ทดสอบ หรือทดสอบและบันทึกผลการทดสอบโปรแกรมได้โดยอัตโนมัติ

เครื่องมือนี้สามารถนำข้อมูลผลการทดสอบมาประมวลผล และออกรายงานสรุปผลการทดสอบ และรายงานความก้าวหน้าในการทดสอบให้กับผู้ใช้ นอกจากนี้ผู้ใช้อย่างยังสามารถกำหนดรูปแบบ และข้อมูลที่จะแสดงในรายงานได้ อีกด้วย

ระบบนี้สามารถทดสอบโปรแกรมที่เขียนด้วยภาษาซี (C Language) และทำงานบนระบบปฏิบัติการยูนิกซ์ (UNIX) ซึ่งทำให้การพัฒนาโปรแกรมโดยใช้ภาษาอื่นไม่สามารถทำงานแบบอัตโนมัติได้

2.6.2) Test Director 6

ถูกพัฒนาโดย บริษัท Mercury Interactive เป็นเครื่องมือที่ช่วยองค์กรในการทดสอบซอฟต์แวร์ทุกขั้นตอน โดยเริ่มจากการวางแผนการทดสอบ การดำเนินการทดสอบ และการจัดการข้อผิดพลาดที่เกิดขึ้น ในขั้นตอนการวางแผนการทดสอบ

เครื่องมือนี้จะช่วยสร้างแผนการทดสอบโดยผู้ใช้งานสามารถสร้างแผนการทดสอบในไมโครซอฟต์เวิร์ด (Microsoft Word) และนำเข้ามาใช้ในเครื่องมือนี้ได้ทันที จากนั้นผู้ทดสอบจะเป็นผู้สร้าง และบันทึกกรณีทดสอบเข้าสู่ระบบ ในส่วนของการดำเนินการทดสอบ เครื่องมือดังกล่าวสามารถทำการทดสอบโดยอัตโนมัติ หรือให้ผู้ทดสอบดำเนินการทดสอบเองแล้วรายงานผลการทดสอบกลับมายังเครื่องมือ หากเกิดข้อผิดพลาด เครื่องมือสามารถให้ข้อมูลที่เป็นประโยชน์ในการวิเคราะห์ข้อผิดพลาดเพื่อหาแนวทางแก้ไข นอกจากนี้เครื่องมือยังสามารถออกรายงานสรุปผลการทดสอบตามรูปแบบที่ผู้ใช้กำหนดโดยส่งข้อมูลไปยังไมโครซอฟต์เอ็กเซล (Microsoft Excel) และไมโครซอฟต์เวิร์ด (Microsoft Word) ได้อีกด้วย

2.6.3) พลัสวันเทส (+1 TEST)

เป็นระบบจัดการกรณีทดสอบ ของบริษัทพลัสวันซอฟต์แวร์เอ็นจิเนียริ่ง (+1 Software Engineering Co. Ltd.) ซึ่งสามารถใช้ได้กับทุกขั้นตอนในการทดสอบโปรแกรมทั้งการทดสอบระดับหน่วย การทดสอบการรวม การทดสอบย้อนกลับ และการทดสอบระบบ โดยระบบจะสร้างบทคำสั่งการทดสอบ เพื่อให้ผู้ทดสอบนำไปทดสอบโปรแกรมที่พัฒนา และนำผลการทดสอบมาทำการบันทึก เข้าสู่ระบบเพื่อที่ระบบจะนำผลการทดสอบมาสรุป และออกรายงานผลการทดสอบต่อไป ผู้ใช้สามารถเพิ่ม-ลด หรือแก้ไขข้อมูลกรณีทดสอบที่ระบบบันทึกไว้ได้ ระบบพลัสวันเทสสามารถสร้างบททดสอบโปรแกรมที่พัฒนาขึ้นด้วยภาษาซี และทำงานบนเครื่องชั้นเวิร์กสเตชัน (SUN Workstation) บนระบบปฏิบัติการโซลาริส (Solaris 1.x and 2.x)

ในขณะนี้ยังไม่มีเครื่องมือที่สามารถช่วยผู้ทดสอบในการสร้างกรณีทดสอบด้วยวิธีการแบบแบล็คบ็อก ผู้วิจัยจึงได้พัฒนาเครื่องมือที่สามารถช่วยในการสร้างกรณีทดสอบแบบแบล็คบ็อกขึ้นเพื่อช่วยลดภาระของผู้ทดสอบ